

Analysis of Airbnb Pricing Patterns in New York

Tanisha Dutta - tdutta@ucsb.edu

2026-06-15

Abstract

Airbnb prices can vary widely across listings, even within the same city. Some listings are priced much higher because they offer more space, are located in popular neighborhoods, or provide an entire private place instead of a shared room. In this project, I studied Airbnb listing prices in New York using exploratory data analysis and statistical learning methods.

The goal of the project was to understand which listing characteristics are most related to price and to build models that can predict listing price using available listing information. For my analysis, I used Airbnb listing data from Kaggle and focused on New York listings to avoid comparing prices across cities with different local currencies. After cleaning the data, I examined pricing patterns by room type, property type, neighborhood, number of bedrooms, and number of guests accommodated. Then, I applied supervised learning methods to predict listing price using these features.

My results show that Airbnb prices in New York are closely related to listing size, room type, and location. Listings with more bedrooms or higher guest capacity tend to have higher prices, and entire places are much more expensive than private or shared rooms. Several Manhattan neighborhoods also have higher median prices than other areas. Overall, my project shows that Airbnb pricing is influenced by a combination of listing characteristics and the neighborhood.

1. Introduction

Airbnb has become a major part of the short-term rental market in cities around the world. In a city like New York, listings differ so much that prices are not uniform. A small private room in one neighborhood are often much cheaper than an entire apartment in another area, even if both listings are in the same city.

I wanted to use this dataset to study what factors are most related to Airbnb listing prices in New York. Last summer, when I interned in New York, I was having a difficult time finding an affordable Airbnb in Midtown, which is where my office was, but I found several properties in Brooklyn and in the outskirts, which made me curious about this topic. The main question I focus on is: which listing characteristics help explain Airbnb price, and how accurately can price be predicted using statistical learning methods?

The dataset includes listing features such as room type, property type, neighborhood, number of bedrooms, number of guests accommodated, booking rules, and host information. These variables make it possible to look at which features matter the most instead of only asking whether a model can predict price.

Past studies of Airbnb and short-term rental markets have been conducted and have often focused on pricing, location, reviews, and host/listing characteristics. This project follows a similar idea by using statistical learning to study Airbnb pricing patterns but specifically in New York.

This report is structured for readability, so I only included the most essential plots and graphs in the 9-page report and included an appendix beginning Page 10 with the entire rest of my code and outputs shown. The methods I used in this project include exploratory data analysis and supervised learning. First, I examined the distribution of prices and compared median prices across room types, property types, neighborhoods,

number of bedrooms, and guest capacity. Then, I used regression models to predict listing price. Since the price variable is right-skewed, I used the logarithm of price for modeling while keeping the original price variable for easier interpretation in graphs and summaries.

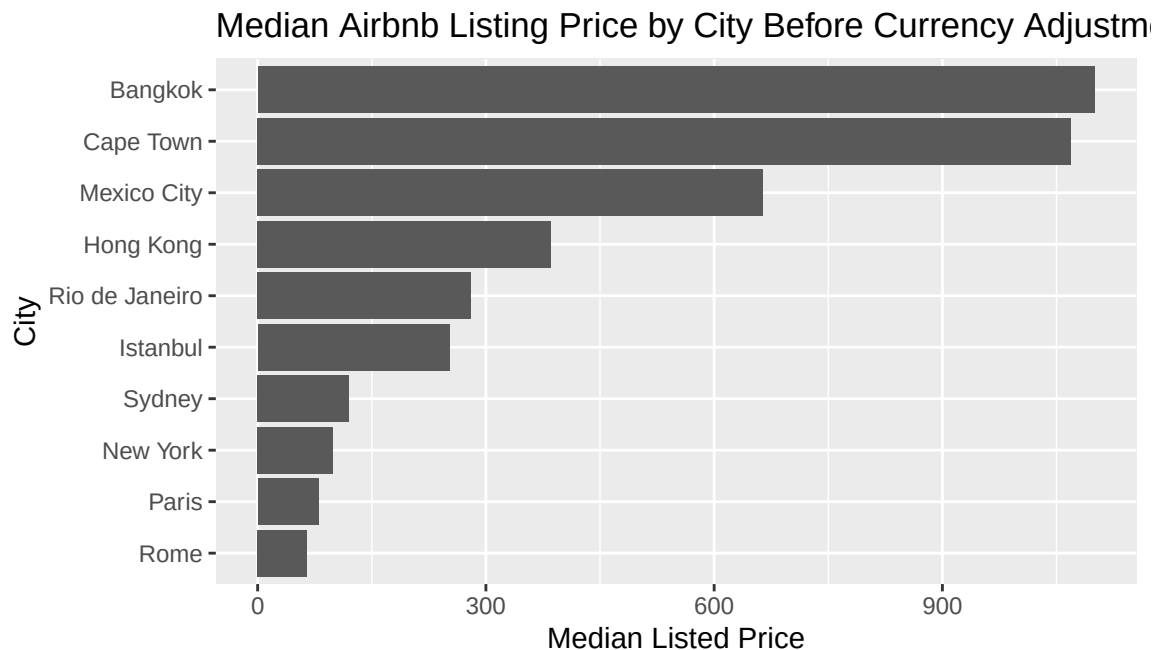
One important discovery from the early data exploration was that the original dataset included multiple cities, but the prices did not appear to be recorded in one common currency. Because comparing raw prices across cities could be misleading, I limited the final analysis to New York listings. Within the New York data, the main patterns showed that price increases with listing size and capacity, and that room type and neighborhood are also strongly related to price. These findings provide a clear basis for the prediction models used later in the project.

2. Data

For this project, I used the Airbnb Listings dataset from Kaggle. The dataset contains listing-level information for Airbnb rentals across multiple major cities. The original dataset contained 279,712 listings and 33 variables. Each row represents one Airbnb listing, and the columns describe features such as listing name, host information, location, property type, room type, price, minimum nights, maximum nights, review scores, and booking options.

The dataset was downloaded from Kaggle: <https://www.kaggle.com/datasets/ulrikthgepedersen/airbnb-listings/da>

During the initial exploration, I found that the dataset included listings from 10 cities: Paris, New York, Sydney, Rome, Rio de Janeiro, Istanbul, Mexico City, Bangkok, Cape Town, and Hong Kong. At first, I considered using all cities together. However, when I compared median prices by city, the results did not appear directly comparable.



The figure above shows the median listed price by city before any currency adjustment. Cities such as Bangkok, Cape Town, and Mexico City had much higher numerical median prices than cities such as New York, Paris, and Rome. This pattern shows that the price variable may have been recorded in each city's local currency rather than converted into one common currency. Since this project focuses on predicting and interpreting listing price, comparing raw prices across cities could have led to misleading conclusions.

For this reason, I restricted the final analysis to New York listings only. This allowed the project to focus on pricing patterns within a single city, where the listed prices were more directly comparable. New York was

also a good choice because it had one of the largest sample sizes in the dataset and included a wide range of neighborhoods, room types, and property types.

Table 1: Table 1. Dataset size before and after cleaning

Dataset Stage	Number of Listings	Number of Variables
Original Airbnb dataset	279712	33
New York subset before cleaning	37012	33
Final modeling dataset	33044	16

The New York subset contained 37,012 listings and 33 variables before cleaning. Before modeling, I cleaned and preprocessed the data. I removed listings with missing prices, prices below \$10, and extreme high prices above the 99th percentile to remove unrealistic or highly unusual listings that could strongly affect the model. After cleaning, the price variable ranged from \$10 to \$850, with a median price of \$95 and a mean price of \$125.30. Since the mean was still higher than the median, the price distribution was slightly right-skewed. Due to this, I used the logarithm of price as the main response variable for the regression models, while keeping the original price variable for descriptive summaries and visualizations.

The New York data originally included 220 neighborhoods, 72 property types, and 4 room types. Since neighborhood and property type had many categories, I simplified these variables before modeling. The most common neighborhoods and property types were kept as separate categories, while less frequent categories were grouped into an “Other” category. This made the modeling process easier to interpret and helped avoid creating too many sparse categories. After selecting the final variables and removing incomplete rows, the final modeling dataset contained 33,044 listings and 16 variables.

The response variable in this project is listing price, represented by both the original `price` variable and the transformed `log_price` variable. The main predictor variables include room type, simplified property type, simplified neighborhood, number of guests accommodated, number of bedrooms, minimum nights, maximum nights, host listing count, host verification status, instant booking status, and geographic coordinates. I mainly chose to study this dataset to understand which Airbnb listing characteristics are most related to price and to evaluate how well statistical learning models can predict listing prices in New York.

3. Methodology

In this project, I used exploratory data analysis and supervised learning methods to study Airbnb listing prices in New York. I start by cleaning the data cleaning producing descriptive summaries, followed by predictive modeling using three supervised learning methods: multiple linear regression, regression tree, and random forest regression.

3.1 Data Preprocessing

The original dataset contained listings from multiple cities, but the final analysis was restricted to New York listings. I made this decision because the price variable appeared to be recorded in local currencies rather than a single standardized currency. Restricting the analysis to one city made the price values more comparable and allowed the project to focus on within-city pricing patterns.

I cleaned the data by removing listings with missing values in the variables used for modeling. I removed prices below \$10 because they were likely unrealistic or not meaningful for an Airbnb listing. I also removed prices above the 99th percentile to reduce the influence of extreme outliers. The final cleaned price range was \$10 to \$850.

The original price distribution was right-skewed, meaning that most listings were concentrated at lower prices while a smaller number of expensive listings pulled the distribution to the right. Because of this, I used the logarithm of price as the response variable in the regression models to make them more stable.

I also simplified high-cardinality categorical variables (factors with many categories). I kept the most common property types and neighborhoods as their own categories and grouped the rest into an “Other” category. This reduced the number of sparse categories and made the models easier to interpret.

Finally, I split the cleaned data into training and test sets. The training set contained 80% of the data and was used to fit the models. The test set contained the remaining 20% and was used to evaluate prediction performance on listings that were not used during training.

3.2 Exploratory Data Analysis

Exploratory data analysis helped understand the main pricing patterns before fitting the models. I examined the distribution of price and log price, then compared median prices across room type, property type, neighborhood, number of bedrooms, and number of guests accommodated. These summaries helped identify which variables appeared most strongly related to Airbnb price.

The exploratory analysis used histograms, bar charts, and line plots. My goal was not only to describe the data, but also to provide context for the supervised learning models. If certain variables showed strong differences in median price during exploration, I assumed they would be useful predictors in the models.

3.3 Multiple Linear Regression

The first supervised learning model I tried was multiple linear regression. I used this as an interpretable baseline model for predicting log price. The model predicts log price as a linear combination of listing, host, and location characteristics. The general mathematical form of the model is:

$$\log(\text{price}) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \epsilon$$

where $(X_1, X_2, \dots, X_p)$ represent the predictor variables, including room type, property type, neighborhood, accommodates, bedrooms, booking rules, host characteristics, latitude, and longitude.

The linear regression model is useful because the estimated coefficients show the direction and approximate size of the relationship between each predictor and log price, while holding the other variables constant. However, I know that linear regression may not fully capture nonlinear relationships or complex interactions between features, so I moved on to other models.

3.4 Regression Tree

The second supervised learning model I tried was a regression tree. A regression tree predicts a continuous response variable by recursively splitting the data into smaller groups. At each split, the model chooses a variable and split point that reduces prediction error within the resulting groups.

The regression tree used the same response variable and predictors as the linear regression model. The response variable was log price, and the predictors included neighborhood, property type, room type, accommodates, bedrooms, booking restrictions, host characteristics, and geographic coordinates.

I tried a regression tree since it provides a simple rule-based structure for understanding pricing patterns. For example, the tree can show whether property type, room type, bedrooms, or neighborhood creates major differences in predicted price. However, I also know that a single tree can be sensitive to the training data and may not always provide the best prediction accuracy, and the results were not strong, so I also tried random forests.

3.5 Random Forest Regression

The third supervised learning model I tried was random forest regression. A random forest is an ensemble method that builds many regression trees and averages their predictions. Unlike a single regression tree, which depends on one tree structure, a random forest combines many trees to improve prediction accuracy and reduce overfitting.

For this project, the random forest used 100 trees and considered 4 randomly selected variables at each split for ease of time. It used the same response variable and predictors as the linear regression and regression tree models. I tried random forests to capture nonlinear relationships and interactions between variables more flexibly than the other two models. Although random forests are usually less directly interpretable than linear regression or a single tree, they provide variable importance measures. These measures helped me identify which predictors contribute most to prediction accuracy.

3.6 Model Evaluation

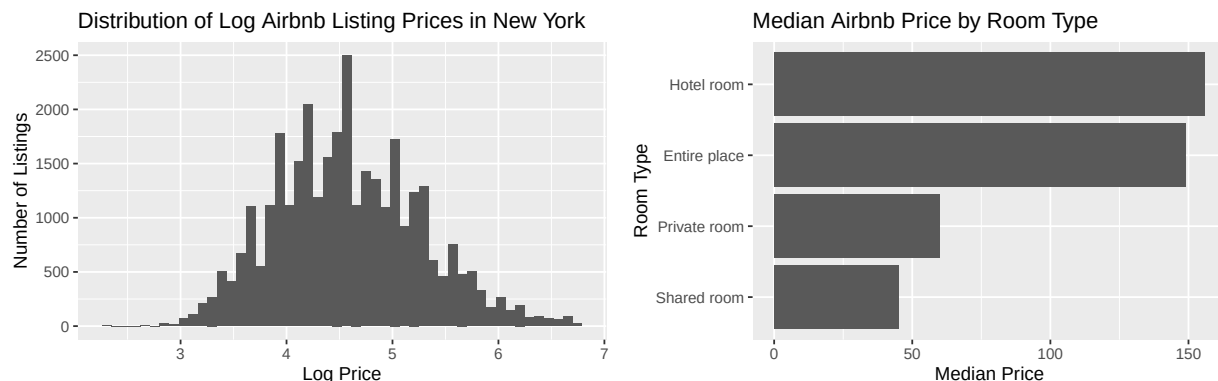
All three models were evaluated on the same test set. Model performance was measured using root mean squared error, mean absolute error, and (R^2). RMSE measures the typical size of prediction errors and gives more weight to large errors. MAE measures the average absolute difference between predicted and actual prices and is easier to interpret on the original dollar scale. (R^2) measures the proportion of variation in log price explained by the model.

Because the models were trained on log price, I first calculated RMSE, MAE, and (R^2) on the log-price scale, then converted predicted log prices back to the original price scale so that RMSE and MAE could also be interpreted in dollars.

4. Results

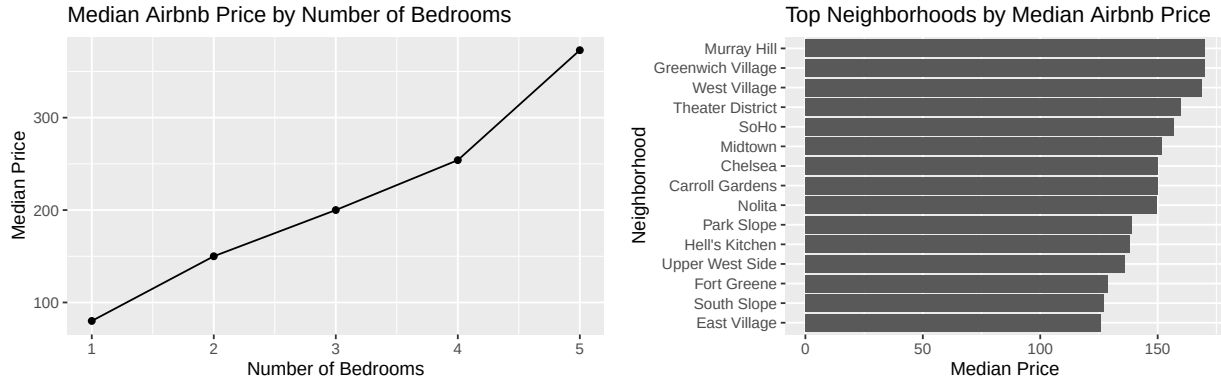
4.1 Exploratory Data Analysis

After cleaning the New York Airbnb listings data, my final modeling dataset contained 33,044 listings and 16 variables. The cleaned price variable ranged from \$10 to \$850, with a median of \$95 and a mean of \$125.30. Since the mean was higher than the median, the price distribution was still right-skewed. Due to this, I used the original price variable for descriptive summaries and the logarithm of price as the response variable for modeling.



The two figures above show the distribution of log prices and the median price by room type. The log-transformed price variable is more balanced than the original price variable, which supports using log price for modeling. Room type was also strongly related to price. Hotel rooms had the highest median price at \$156, followed closely by entire places at \$149. Private rooms had a much lower median price of \$60, while shared rooms had the lowest median price of \$45. This suggests that guests pay more for privacy and full access to a listing.

A similar pattern appeared when comparing price by number of bedrooms. One-bedroom listings were the most common and had a median price of \$80. As the number of bedrooms increased, the median price also increased. Two-bedroom listings had a median price of \$150, three-bedroom listings had a median price of \$200, four-bedroom listings had a median price of \$254, and five-bedroom listings had a median price of \$373. This shows that listing size is strongly related to price.



The two figures above show that listing size and neighborhood were also important pricing factors. Median price generally increased as the number of bedrooms increased. One-bedroom listings had a median price of \$80, while two-bedroom listings had a median price of \$150, three-bedroom listings had a median price of \$200, and five-bedroom listings had a median price of \$373. Neighborhood differences were also clear. Greenwich Village, Murray Hill, West Village, Theater District, SoHo, Midtown, Chelsea, and Hell's Kitchen were among the neighborhoods with the highest median prices. These areas are generally central or high-demand neighborhoods, which helps explain their higher prices.

Overall, the exploratory analysis suggests that Airbnb price is closely related to room type, listing size, and location. Entire or hotel-style listings, larger listings, and listings in central neighborhoods tended to have higher median prices. These patterns provide a strong basis for the supervised learning models.

4.2 Predictive Modeling Results

After the exploratory analysis, I fit three supervised learning models to predict log Airbnb price: multiple linear regression, regression tree, and random forest regression. I used the same training and test split for each model, with 80% of the cleaned data used for training and 20% used for testing. They also used the same response variable, log price, and the same set of predictors.

The linear regression model provided an interpretable baseline. On the test data, the linear regression model had an RMSE of 0.4430 and an MAE of 0.3399 on the log-price scale. When predictions were converted back to the original price scale, the RMSE was about \$77.82 and the MAE was about \$43.72. The test-set (R^2) was 0.5687, meaning that the model explained about 56.9% of the variation in log Airbnb price on the test data.

The regression tree model was less accurate than the linear regression model, but it was still useful for interpretation. The first major split in the tree was based on property type, separating entire-property and hotel-style listings from private-room, shared-room, and other lower-priced property categories. The tree also used neighborhood, bedrooms, accommodates, host listing count, and longitude to split listings into different predicted price groups. On the test data, the regression tree had an RMSE of 0.4747 and an MAE of 0.3659 on the log-price scale. On the original price scale, the tree had an RMSE of about \$84.02 and an MAE of about \$47.30. The test-set (R^2) was 0.5048.

The random forest model performed best overall. Random forest regression is more flexible than a single regression tree because it builds many trees and averages their predictions. This allows the model to capture nonlinear relationships and interactions between listing characteristics. On the test data, the random forest had an RMSE of 0.3892 and an MAE of 0.2903 on the log-price scale. On the original price scale, the RMSE was about \$69.88 and the MAE was about \$38.22. The test-set (R^2) was 0.6672, meaning that the random forest explained about 66.7% of the variation in log Airbnb price.

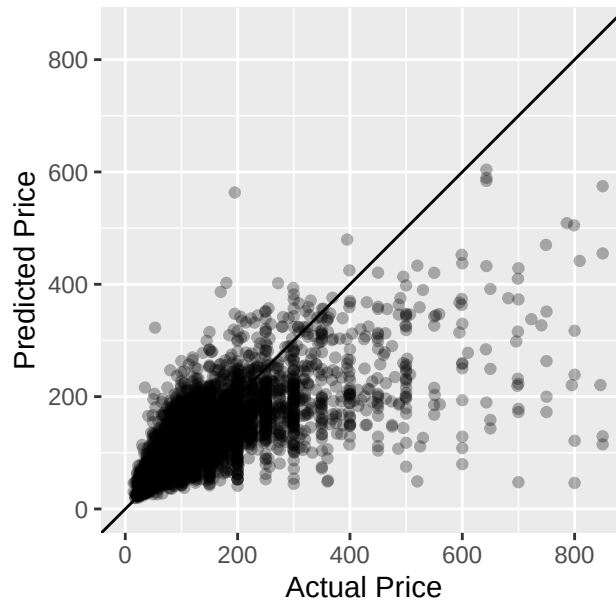
Table 2: Table 2. Model comparison on the test data

Model	RMSE Log	MAE Log	RMSE Price	MAE Price	Test R-Squared
Linear Regression	0.4430	0.3399	77.8185	43.7162	0.5687
Regression Tree	0.4747	0.3659	84.0209	47.3010	0.5048
Random Forest	0.3892	0.2903	69.8816	38.2171	0.6672

Table 2 compares the three supervised learning models. The random forest had the lowest RMSE and MAE and the highest test-set (R^2), making it the best predictive model. The random forest reduced the average absolute prediction error to about \$38.22, compared with \$43.72 for the linear regression model and \$47.30 for the regression tree.

Although the random forest had the best predictive accuracy, the other models were still useful. The linear regression model was easier to interpret because its coefficients showed how each predictor was associated with log price while holding other variables constant. The regression tree was useful because it provided a simple rule-based structure for understanding how listings were grouped into different price levels. The random forest was less directly interpretable, but it achieved the strongest prediction performance.

Random Forest: Actual vs Predicted Airbnb Prices



The figure above shows the actual versus predicted prices for the random forest model. Points closer to the diagonal line represent more accurate predictions. Many listings fall near the diagonal line, which means that the model captured many of the major pricing patterns in the data. However, the model still had difficulty predicting some of the most expensive listings. This might be due to the fact that high-priced listings may depend on details that were not in the dataset, such as certain amenities, interior design, nearby attractions, etc.

Table 3: Table 3. Top random forest variable importance scores

Variable	Percent Increase in MSE	Increase in Node Purity
host_total_listings_count	58.59	697.37
latitude	38.95	1061.49
accommodates	38.70	1316.30

Variable	Percent Increase in MSE	Increase in Node Purity
neighbourhood_simplified	37.65	1286.42
bedrooms	33.82	775.49
longitude	30.63	1260.85
minimum_nights	27.63	289.65
maximum_nights	25.09	372.90
host_is_superhost	22.42	85.51
property_type_simplified	22.16	2817.97

Table 3 shows the most important predictors in the random forest model based on percent increase in mean squared error. Host total listings count had the highest importance score, followed by latitude, accommodates, neighborhood, bedrooms, and longitude. This shows that both host-related features and location-based features contributed strongly to prediction accuracy. Listing size variables such as accommodates and bedrooms were also important, which supports my previous observation that larger listings usually have higher prices.

Overall, the modeling results support my main findings from the exploratory analysis. Airbnb prices in New York are influenced by a combination of location, listing size, property type, room type, and host characteristics. The random forest model performed best because it was able to capture more complex relationships between these variables than the linear regression model or single regression tree.

Conclusion and Future Study

In this project, I studied Airbnb pricing patterns in New York using exploratory data analysis and supervised statistical learning methods. The original dataset contained listings from multiple cities, but the final analysis focused only on New York because the raw price values across cities did not appear to be recorded in a common currency. After cleaning and preprocessing the data, the final modeling dataset contained 33,044 listings and 16 variables.

My initial exploratory analysis showed that Airbnb prices in New York are closely related to listing size, room type, property type, and location. Listings with more bedrooms or higher guest capacity generally had higher median prices. Hotel rooms and entire places were much more expensive than private rooms and shared rooms, showing that privacy and full-property access contribute heavily to pricing. Neighborhood differences were also clear, with central or high-demand neighborhoods such as Greenwich Village, Murray Hill, West Village, Theater District, SoHo, Midtown, Chelsea, and Hell’s Kitchen having some of the highest median prices.

The supervised learning results supported these exploratory findings. I compared three models: multiple linear regression, regression tree, and random forest regression. The random forest model performed best, with a test-set (R^2) of 0.6672 and a mean absolute error of about \$38.22 on the original price scale. This means that the model explained about 66.7% of the variation in log Airbnb price and was typically within about \$38 of the actual listing price. The linear regression and regression tree models were less accurate, but they were still useful for interpretation. Across all three models, the most important pricing factors were location, listing size, room type, property type, and host-related characteristics.

The most important discovery I made from this project is that Airbnb price is not determined by one factor alone. Instead, price is influenced by a combination of physical listing characteristics, location-based demand, and host-related features. Larger listings, full-property listings, and listings in central neighborhoods tended to have higher prices. Statistical learning methods can be useful not only for predicting Airbnb prices, but also for understanding the main factors that drive pricing differences across listings.

There are several ways this project could be extended in the future. First, future work could include additional predictive models such as gradient boosting or tuned random forest models to see whether prediction accuracy improves further. Second, amenities could be included more carefully. I did not include the amenities variable

in the main models because it is stored as text/list-style information, which would require additional feature engineering to convert amenities into usable predictor variables. In a future study, amenities such as air conditioning, parking, kitchen access, washer and dryer, or workspace could be extracted and added to the model. Third, review scores could also be included if more complete review information were available. In the current dataset, many review score variables had missing values, so including them would have removed a large number of listings or required additional imputation. If more reviews were gathered or missing review information were handled carefully, review scores or review text could help study whether guest satisfaction is related to price. Finally, future analysis could compare New York with other cities after converting all prices into a common currency, which would make cross-city pricing comparisons more meaningful.

Overall, this project shows that Airbnb prices in New York are shaped by a combination of size, privacy, location, and host characteristics. Among the models tested, the random forest provided the strongest prediction performance, while the exploratory analysis and simpler models helped explain why certain listings tend to be more expensive than others.

References

Ichiba, T. (2026). PSTAT 135: Big Data Analytics lecture notes. University of California, Santa Barbara.

https://uc-r.github.io/regression_trees

<https://www.kaggle.com/datasets/ulrikthgepedersen/airbnb-listings/data>

<https://medium.com/inst414-data-science-tech/predicting-airbnb-stay-prices-using-ridge-regression-9d8bac9e5ec7>

Appendix

The appendix includes the R code used for data cleaning, exploratory data analysis, model fitting, model evaluation, and visualization. The main report hides code for readability, while this appendix provides the full code for reproducibility.

Explore Data

```
dim(listings)
```

```
## [1] 279712      33
```

```
names(listings)
```

```
## [1] "listing_id"           "name"
## [3] "host_id"             "host_since"
## [5] "host_location"       "host_response_time"
## [7] "host_response_rate"  "host_acceptance_rate"
## [9] "host_is_superhost"   "host_total_listings_count"
## [11] "host_has_profile_pic" "host_identity_verified"
## [13] "neighbourhood"       "district"
## [15] "city"                "latitude"
## [17] "longitude"           "property_type"
## [19] "room_type"           "accommodates"
## [21] "bedrooms"            "amenities"
## [23] "price"               "minimum_nights"
## [25] "maximum_nights"      "review_scores_rating"
## [27] "review_scores_accuracy" "review_scores_cleanliness"
## [29] "review_scores_checkin" "review_scores_communication"
## [31] "review_scores_location" "review_scores_value"
## [33] "instant_bookable"
```

```
head(listings)
```

```
##   listing_id           name  host_id
## 1    281420 Beautiful Flat in le Village Montmartre, Paris 1466919
## 2    3705183              39 mÃ² Paris (Sacre CÃ"ur) 10328771
## 3    4082273    Lovely apartment with Terrace, 60m2 19252768
## 4    4797344    Cosy studio (close to Eiffel tower) 10668311
## 5    4823489 Close to Eiffel Tower - Beautiful flat : 2 rooms 24837558
## 6    4898654    NEW - Charming apartment Le Marais    505535
##   host_since           host_location host_response_time host_response_rate
## 1 2011-12-03 Paris, Ile-de-France, France                NA
## 2 2013-11-29 Paris, Ile-de-France, France                NA
## 3 2014-07-31 Paris, Ile-de-France, France                NA
```

```

## 4 2013-12-17 Paris, Ile-de-France, France NA
## 5 2014-12-14 Paris, Ile-de-France, France NA
## 6 2011-04-13 Paris, Ile-de-France, France NA
## host_acceptance_rate host_is_superhost host_total_listings_count
## 1 NA f 1
## 2 NA f 1
## 3 NA f 1
## 4 NA f 1
## 5 NA f 1
## 6 NA f 1
## host_has_profile_pic host_identity_verified neighbourhood district city
## 1 t f Buttes-Montmartre Paris
## 2 t t Buttes-Montmartre Paris
## 3 t f Elysee Paris
## 4 t t Vaugirard Paris
## 5 t f Passy Paris
## 6 t t Temple Paris
## latitude longitude property_type room_type accommodates bedrooms
## 1 48.88668 2.33343 Entire apartment Entire place 2 1
## 2 48.88617 2.34515 Entire apartment Entire place 2 1
## 3 48.88112 2.31712 Entire apartment Entire place 2 1
## 4 48.84571 2.30584 Entire apartment Entire place 2 1
## 5 48.85500 2.26979 Entire apartment Entire place 2 1
## 6 48.86428 2.35370 Entire apartment Entire place 2 1
##
## 1 ["Heating", "Kitchen
## 2 ["Shampoo", "Heating", "Kitchen", "Essentials", "Washer
## 3 ["Heating", "TV", "Kitchen
## 4 ["Heating", "TV
## 5 ["Heating", "TV", "Kitchen", "Essentials", "Hair dryer", "Washer", "Dryer", "Bathtub", "Wifi", "El
## 6 ["Heating", "TV", "Kitchen", "Essentials", "Washer", "Smoke alarm",
## price minimum_nights maximum_nights review_scores_rating
## 1 53 2 1125 100
## 2 120 2 1125 100
## 3 89 2 1125 100
## 4 58 2 1125 100
## 5 60 2 1125 100
## 6 95 2 1125 100
## review_scores_accuracy review_scores_cleanliness review_scores_checkin
## 1 10 10 10
## 2 10 10 10
## 3 10 10 10
## 4 10 10 10
## 5 10 10 10
## 6 10 10 10
## review_scores_communication review_scores_location review_scores_value
## 1 10 10 10
## 2 10 10 10
## 3 10 10 10
## 4 10 10 10
## 5 10 10 10
## 6 10 10 10
## instant_bookable
## 1 f

```

```
## 2          f
## 3          f
## 4          f
## 5          f
## 6          f
```

Clean Data

```
# check missing values by column
missing_summary <- listings %>%
  summarise(across(everything(), ~sum(is.na(.)))) %>%
  pivot_longer(cols = everything(),
               names_to = "variable",
               values_to = "missing_count") %>%
  arrange(desc(missing_count))
missing_summary
```

```
## # A tibble: 33 x 2
##   variable      missing_count
##   <chr>          <int>
## 1 host_response_rate      128782
## 2 host_acceptance_rate    113087
## 3 review_scores_value      91785
## 4 review_scores_location   91775
## 5 review_scores_checkin    91771
## 6 review_scores_accuracy   91713
## 7 review_scores_communication 91687
## 8 review_scores_cleanliness 91665
## 9 review_scores_rating     91405
## 10 bedrooms               29435
## # i 23 more rows
```

```
# convert variables
airbnb_explore <- listings %>%
  mutate(
    # remove symbols if any
    price = as.numeric(gsub("$,", "", price)),
    # convert rates to percentages
    host_response_rate = as.numeric(gsub("%", "", host_response_rate)),
    host_acceptance_rate = as.numeric(gsub("%", "", host_acceptance_rate)),
    # convert to date if not already
    host_since = as.Date(host_since),
    # new column to calculate how many years host has been a host
    host_years = as.numeric(difftime(Sys.Date(), host_since, units = "days")) / 365,
    # convert categorical variables to factors
    city = as.factor(city),
    neighbourhood = as.factor(neighbourhood),
    property_type = as.factor(property_type),
    room_type = as.factor(room_type),
    # convert yes/no variables to factors
    host_is_superhost = as.factor(host_is_superhost),
    host_identity_verified = as.factor(host_identity_verified),
    host_has_profile_pic = as.factor(host_has_profile_pic),
    instant_bookable = as.factor(instant_bookable)
```

```
)
```

Price Distribution Exploration

```
# clean price variable (remove symbols if any)
airbnb_explore <- listings %>%
  mutate(
    price = as.numeric(gsub("$", "", price))
  )

dim(airbnb_explore)
```

```
## [1] 279712      33
```

```
summary(airbnb_explore$price)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      0.0   75.0   150.0   608.8   474.0 625216.0
```

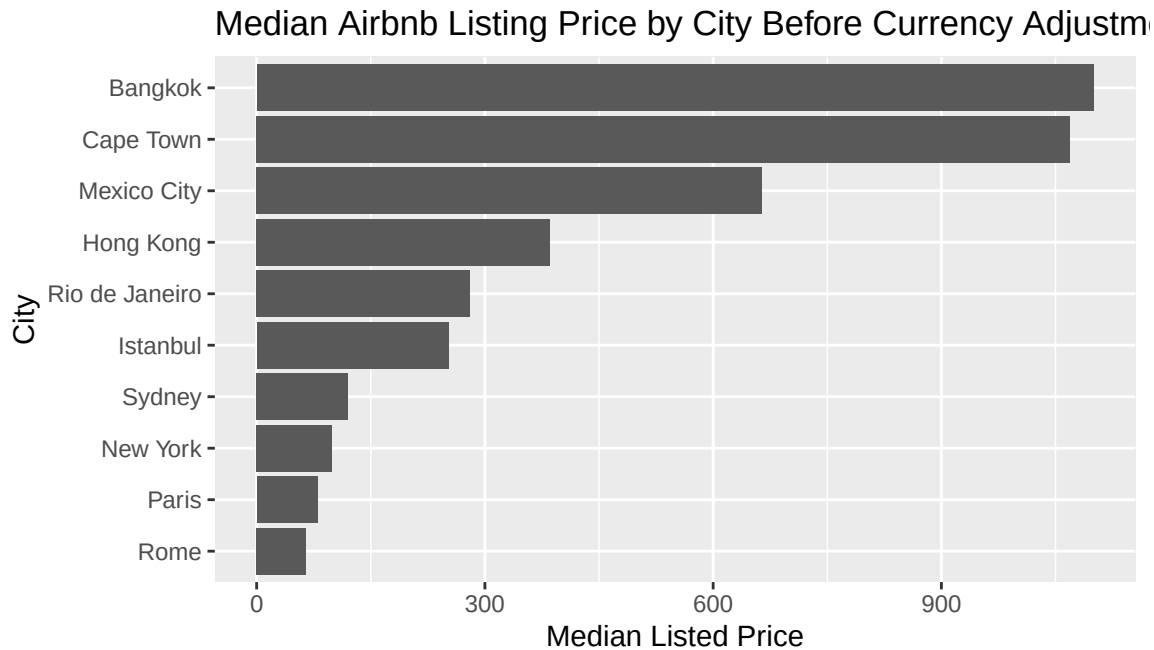
```
# check price distribution by city
price_by_city <- airbnb_explore %>%
  filter(!is.na(price), price > 0) %>%
  group_by(city) %>%
  summarise(
    count = n(),
    median_price = median(price),
    average_price = mean(price),
    min_price = min(price),
    max_price = max(price)
  ) %>%
  arrange(desc(median_price))
price_by_city
```

```
## # A tibble: 10 x 6
```

```
##   city          count median_price average_price min_price max_price
##   <chr>         <int>      <dbl>      <dbl>      <dbl>      <dbl>
## 1 Bangkok      19360        1100        2078.        241      300177
## 2 Cape Town    19086        1069        2405.        122      180000
## 3 Mexico City  20058         664        1150.        100     499000
## 4 Hong Kong     7087         386         746.          1       83665
## 5 Rio de Janeiro 26608         280         743.          33     625216
## 6 Istanbul     24517         252         533.          31     179532
## 7 Sydney       33630         120         222.           5       28613
## 8 New York     36984          99         143.          10       10000
## 9 Paris        64628          80         113.           8       12000
## 10 Rome        27641          65         105.           7       10571
```

```
# plot price distribution by city
price_by_city %>%
  ggplot(aes(x = reorder(city, median_price), y = median_price)) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Median Airbnb Listing Price by City Before Currency Adjustment",
    x = "City",
```

```
y = "Median Listed Price"
)
```



The currency does not seem to be standardized, since New York and Paris median prices are much lower than cities like Bangkok, Mexico City, and Istanbul. Since the comparison would be unfair, we can move ahead with analysis of only one city. I choose New York since the currency is easier for me to analyze and I am more familiar with how the different neighbourhoods in New York vastly affect property pricing, so I think it would be interesting to visualize.

Filter to only New York listings

```
airbnb_ny <- airbnb_explore %>%
  filter(city == "New York")

dim(airbnb_ny)
```

```
## [1] 37012    33
```

```
summary(airbnb_ny$price)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      0.0   60.0   99.0  142.8  151.0 10000.0
```

Clean New York listings dataset

```
airbnb_ny_clean <- airbnb_ny %>%
  mutate(
    # convert rates to percentages
    host_response_rate = as.numeric(gsub("%", "", host_response_rate)),
    host_acceptance_rate = as.numeric(gsub("%", "", host_acceptance_rate)),
    # convert to date if not already
    host_since = as.Date(host_since),
```

```

# new column to calculate how many years host has been a host
host_years = as.numeric(difftime(Sys.Date(), host_since, units = "days")) / 365,
# convert categorical variables to factors
city = as.factor(city),
neighbourhood = as.factor(neighbourhood),
property_type = as.factor(property_type),
room_type = as.factor(room_type),
# convert yes/no variables to factors
host_is_superhost = as.factor(host_is_superhost),
host_has_profile_pic = as.factor(host_has_profile_pic),
host_identity_verified = as.factor(host_identity_verified),
host_has_profile_pic = as.factor(host_identity_verified),
instant_bookable = as.factor(instant_bookable)
) %>%
filter(
  !is.na(price),
  price >= 10,
  price <= quantile(price, 0.99, na.rm = TRUE),
  !is.na(room_type),
  !is.na(property_type),
  !is.na(accommodates),
  !is.na(bedrooms),
  !is.na(minimum_nights),
  !is.na(maximum_nights),
  !is.na(latitude),
  !is.na(longitude),
  !is.na(host_total_listings_count)
)

```

The price is heavily skewed to the right, so we can use a log price variable to use for better results when fitting a linear regression model.

Log-Price Variable

```

airbnb_ny_clean <- airbnb_ny_clean %>%
  mutate(log_price = log(price))

summary(airbnb_ny_clean$log_price)

```

```

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  2.303   4.094   4.554   4.581   5.011   6.745

```

Simplifying Categorical Data

```

# check how many categories each factor has
airbnb_ny_clean %>%
  summarise(
    neighbourhoods = n_distinct(neighbourhood),
    property_types = n_distinct(property_type),
    room_types = n_distinct(room_type)
  )

```

```

##      neighbourhoods property_types room_types
## 1                220              72         4

```

```
# explore categories for each factor
```

```
airbnb_ny_clean %>%
```

```
  count(room_type, sort = TRUE)
```

```
##      room_type      n
## 1 Private room 16090
## 2 Entire place 16037
## 3 Shared room   678
## 4 Hotel room   239
```

```
airbnb_ny_clean %>%
```

```
  count(neighbourhood, sort = TRUE) %>%
```

```
  head(20)
```

```
##      neighbourhood      n
## 1 Bedford-Stuyvesant 2594
## 2 Williamsburg      2541
## 3 Harlem             1844
## 4 Bushwick           1605
## 5 Hell's Kitchen     1285
## 6 Upper West Side    1231
## 7 East Village       1143
## 8 Crown Heights      1066
## 9 Midtown            1048
## 10 Upper East Side   1033
## 11 Greenpoint         730
## 12 Chelsea            678
## 13 East Harlem        677
## 14 Astoria            633
## 15 Lower East Side    624
## 16 Washington Heights 596
## 17 West Village       470
## 18 Financial District 410
## 19 Flatbush           407
## 20 Clinton Hill      389
```

```
airbnb_ny_clean %>%
```

```
  count(property_type, sort = TRUE) %>%
```

```
  head(15)
```

```
##      property_type      n
## 1 Entire apartment 12732
## 2 Private room in apartment 11739
## 3 Private room in house 1834
## 4 Private room in townhouse 1024
## 5 Entire house      865
## 6 Entire condominium 831
## 7 Entire townhouse   533
## 8 Entire loft        501
## 9 Shared room in apartment 498
## 10 Private room in condominium 466
## 11 Private room in loft 342
## 12 Room in hotel      292
## 13 Room in boutique hotel 279
## 14 Entire serviced apartment 228
```

```
## 15           Entire guest suite    190
# only keep most common property types and neighborhoods
top_property_types <- airbnb_ny_clean %>%
  count(property_type, sort = TRUE) %>%
  slice_head(n = 15) %>%
  pull(property_type)

top_neighbourhoods <- airbnb_ny_clean %>%
  count(neighbourhood, sort = TRUE) %>%
  slice_head(n = 50) %>%
  pull(neighbourhood)

airbnb_ny_clean <- airbnb_ny_clean %>%
  mutate(
    property_type_simplified = ifelse(
      property_type %in% top_property_types,
      as.character(property_type),
      "Other"
    ),
    neighbourhood_simplified = ifelse(
      neighbourhood %in% top_neighbourhoods,
      as.character(neighbourhood),
      "Other"
    ),
    property_type_simplified = as.factor(property_type_simplified),
    neighbourhood_simplified = as.factor(neighbourhood_simplified)
  )
```

Final Modeling Data

```
airbnb_ny_final <- airbnb_ny_clean %>%
  select(
    price,
    city,
    neighbourhood_simplified,
    property_type_simplified,
    room_type,
    accommodates,
    bedrooms,
    minimum_nights,
    maximum_nights,
    host_total_listings_count,
    host_is_superhost,
    host_identity_verified,
    instant_bookable,
    latitude,
    longitude
  ) %>%
  drop_na() %>%
  mutate(
    log_price = log(price)
```

```
)

dim(airbnb_ny_final)

## [1] 33044    16

summary(airbnb_ny_final$price)

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    10.0   60.0   95.0  125.3  150.0   850.0

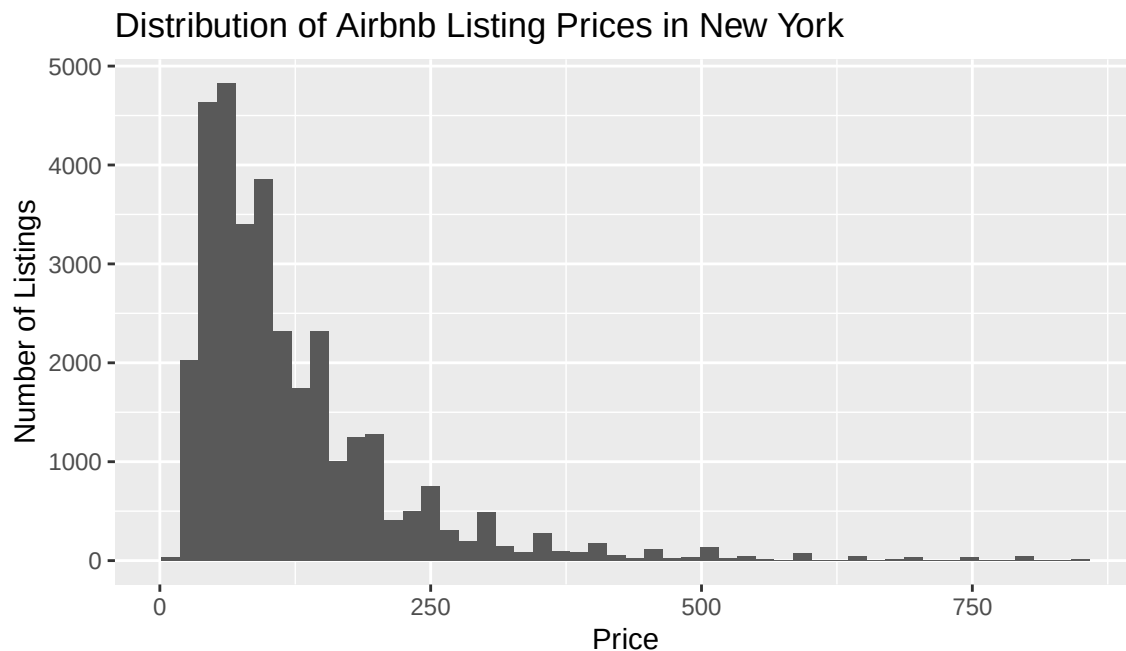
summary(airbnb_ny_final$log_price)

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    2.303   4.094   4.554   4.581   5.011   6.745
```

EDA and Visualizations

Pricing Distribution

```
# pricing distribution
# log pricing distribution
ggplot(airbnb_ny_final, aes(x = price)) +
  geom_histogram(bins = 50) +
  labs(
    title = "Distribution of Airbnb Listing Prices in New York",
    x = "Price",
    y = "Number of Listings"
  )
)
```



```
# log pricing distribution
ggplot(airbnb_ny_final, aes(x = log_price)) +
  geom_histogram(bins = 50) +
  labs(
```

```

title = "Distribution of Log Airbnb Listing Prices in New York",
x = "Log Price",
y = "Number of Listings"
)

```



Median Price by Room Type and Property Type

```

# room type
## summary table
room_type_price <- airbnb_ny_final %>%
  group_by(room_type) %>%
  summarise(
    count = n(),
    median_price = median(price),
    average_price = mean(price)
  ) %>%
  arrange(desc(median_price))
room_type_price

```

```

## # A tibble: 4 x 4
##   room_type    count median_price average_price
##   <fct>      <int>      <dbl>      <dbl>
## 1 Hotel room    239        156        209.
## 2 Entire place 16037        149        177.
## 3 Private room 16090         60         75.1
## 4 Shared room   678         45         69.8

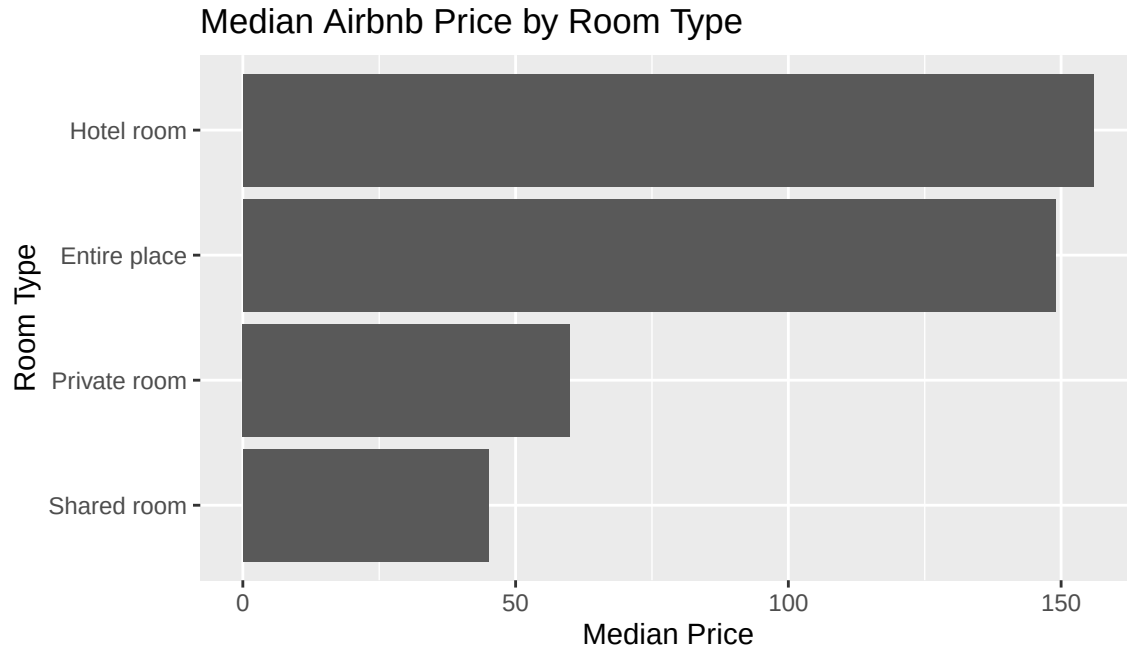
```

```

## plot
room_type_price %>%
  ggplot(aes(x = reorder(room_type, median_price), y = median_price)) +
  geom_col() +
  coord_flip() +

```

```
labs(
  title = "Median Airbnb Price by Room Type",
  x = "Room Type",
  y = "Median Price"
)
```

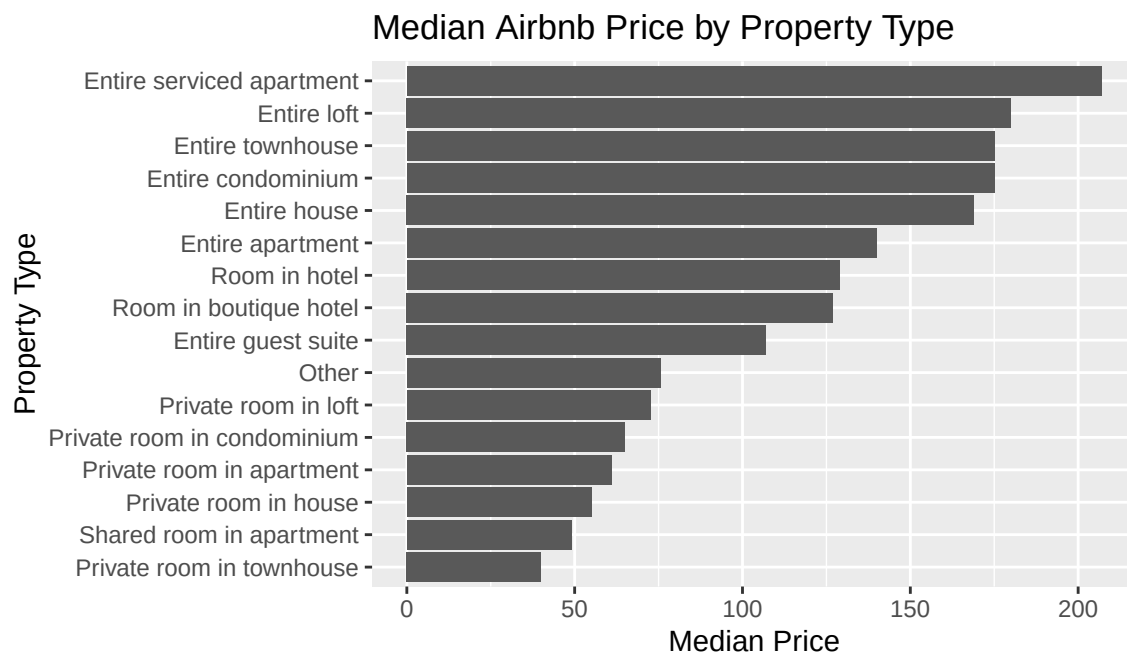


```
# property type
## summary table
property_type_price <- airbnb_ny_final %>%
  group_by(property_type_simplified) %>%
  summarise(
    count = n(),
    median_price = median(price),
    average_price = mean(price)
  ) %>%
  arrange(desc(median_price))
property_type_price
```

```
## # A tibble: 16 x 4
##   property_type_simplified count median_price average_price
##   <fct>                  <int>      <dbl>      <dbl>
## 1 Entire serviced apartment    228        207        255.
## 2 Entire loft                  501        180        236.
## 3 Entire condominium          831        175        217.
## 4 Entire townhouse            533        175        245.
## 5 Entire house                 865        169        217.
## 6 Entire apartment          12732        140        166.
## 7 Room in hotel               292        129        182.
## 8 Room in boutique hotel      279        127        160.
## 9 Entire guest suite          190        107        120.
## 10 Other                     690         75.5        122.
## 11 Private room in loft       342         72.5         86.1
```

```
## 12 Private room in condominium 466 65 84.9
## 13 Private room in apartment 11739 61 73.5
## 14 Private room in house 1834 55 68.5
## 15 Shared room in apartment 498 49 71.6
## 16 Private room in townhouse 1024 40 57.4
```

```
## plot
property_type_price %>%
  ggplot(aes(x = reorder(property_type_simplified, median_price), y = median_price)) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Median Airbnb Price by Property Type",
    x = "Property Type",
    y = "Median Price"
  )
```



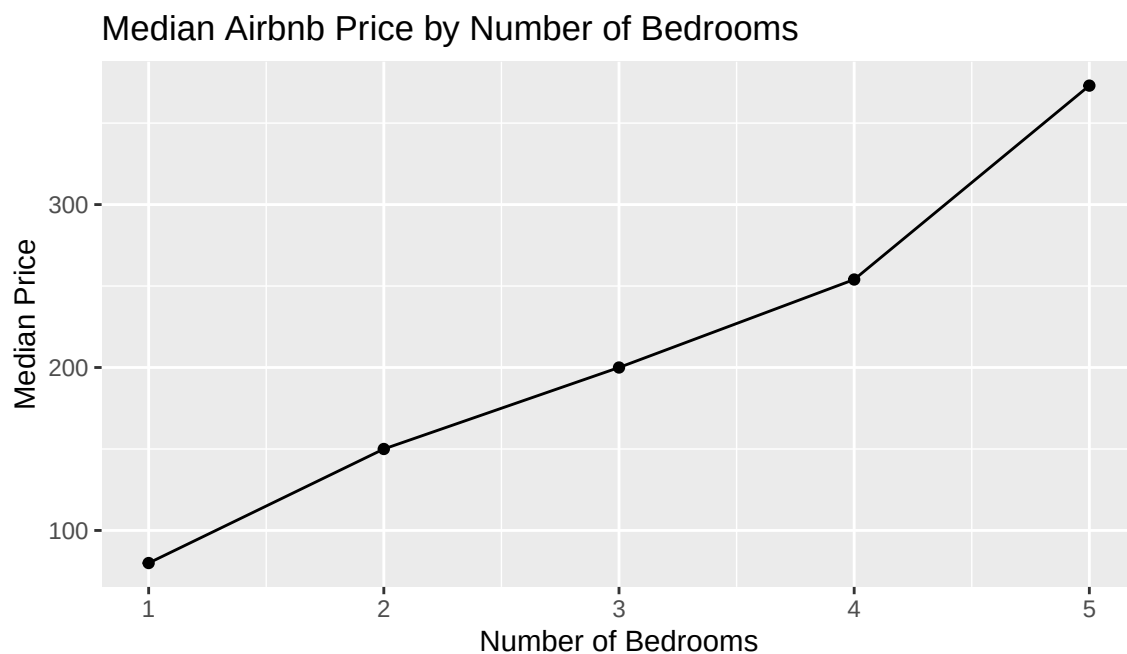
Price vs. Number of Bedrooms

```
## summarise table
bedrooms_price <- airbnb_ny_final %>%
  group_by.bedrooms) %>%
  summarise(
    count = n(),
    median_price = median(price),
    average_price = mean(price)
  ) %>%
  filter(count >= 50)
bedrooms_price
```

```
## # A tibble: 5 x 4
##   bedrooms count median_price average_price
##       <int> <int>         <dbl>         <dbl>
```

```
## 1      1 25932      80      102.
## 2      2  5005     150     188.
## 3      3  1555     200     245.
## 4      4   417     254     310.
## 5      5    88     373     397.
```

```
bedrooms_price %>%
  ggplot(aes(x = bedrooms, y = median_price)) +
  geom_line() +
  geom_point() +
  labs(
    title = "Median Airbnb Price by Number of Bedrooms",
    x = "Number of Bedrooms",
    y = "Median Price"
  )
```



Median Price by Neighbourhoods

```
# top neighbourhoods with most listings
top_neighbourhood_counts <- airbnb_ny_final %>%
  count(neighbourhood_simplified, sort = TRUE) %>%
  filter(neighbourhood_simplified != "Other") %>%
  head(15)

# median price by neighbourhoods (out of the top neighbourhoods)
## summary table
neighbourhood_price <- airbnb_ny_final %>%
  filter(neighbourhood_simplified != "Other") %>%
  group_by(neighbourhood_simplified) %>%
  summarise(
    count = n(),
    median_price = median(price),
```

```

    average_price = mean(price)
  ) %>%
  filter(count >= 100) %>%
  arrange(desc(median_price))
neighbourhood_price %>%
  head(15)

```

```

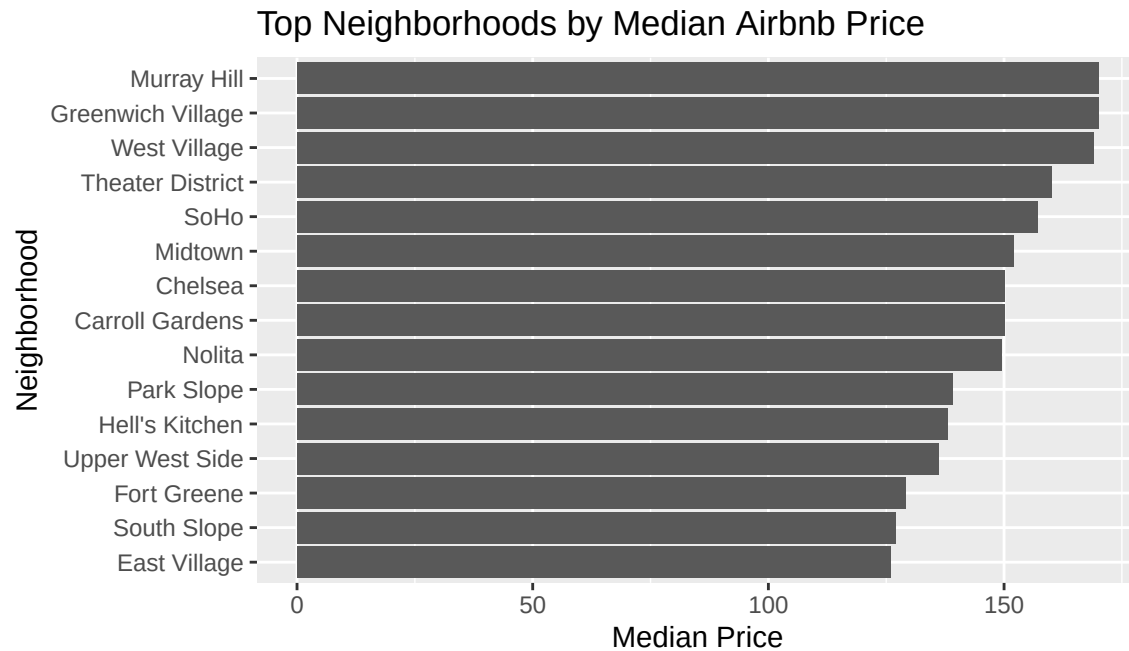
## # A tibble: 15 x 4
##   neighbourhood_simplified count median_price average_price
##   <fct>                <int>      <dbl>      <dbl>
## 1 Greenwich Village      217        170        199.
## 2 Murray Hill            299        170        183.
## 3 West Village           470        169        194.
## 4 Theater District       271        160        192.
## 5 SoHo                   240        157        222.
## 6 Midtown               1048        152        206.
## 7 Carroll Gardens        160        150        175.
## 8 Chelsea                678        150        189.
## 9 Nolita                 186        150.        178.
## 10 Park Slope             323        139        162.
## 11 Hell's Kitchen        1285        138        170.
## 12 Upper West Side       1231        136        163.
## 13 Fort Greene            297        129        151.
## 14 South Slope            196        127        145.
## 15 East Village          1143        126        154.

```

```

## plot
neighbourhood_price %>%
  head(15) %>%
  ggplot(aes(x = reorder(neighbourhood_simplified, median_price), y = median_price)) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Top Neighborhoods by Median Airbnb Price",
    x = "Neighborhood",
    y = "Median Price"
  )

```



Modeling

Split Data into Training and Test Sets

```
set.seed(123)

train_index <- sample(
  1:nrow(airbnb_ny_final),
  size = 0.8 * nrow(airbnb_ny_final)
)

train_data <- airbnb_ny_final[train_index, ]
test_data <- airbnb_ny_final[-train_index, ]

dim(train_data)

## [1] 26435  16

dim(test_data)

## [1] 6609  16
```

Linear Regression Model

```
# fit model
lm_model <- lm(
  log_price ~ neighbourhood_simplified + property_type_simplified +
    room_type + accommodates + bedrooms + minimum_nights +
    maximum_nights + host_total_listings_count +
    host_is_superhost + host_identity_verified +
    instant_bookable + latitude + longitude,
  data = train_data
```

```

)
summary(lm_model)

##
## Call:
## lm(formula = log_price ~ neighbourhood_simplified + property_type_simplified +
##     room_type + accommodates + bedrooms + minimum_nights + maximum_nights +
##     host_total_listings_count + host_is_superhost + host_identity_verified +
##     instant_bookable + latitude + longitude, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.50184 -0.28881 -0.01901  0.25492  2.73851
##
## Coefficients:
##                                     Estimate Std. Error
## (Intercept)                       -6.889e+01  8.141e+00
## neighbourhood_simplifiedBedford-Stuyvesant -1.929e-02  2.300e-02
## neighbourhood_simplifiedBushwick          -6.950e-02  2.421e-02
## neighbourhood_simplifiedCarroll Gardens    2.866e-01  4.440e-02
## neighbourhood_simplifiedChelsea           4.581e-01  2.840e-02
## neighbourhood_simplifiedChinatown          2.647e-01  3.855e-02
## neighbourhood_simplifiedClinton Hill       2.356e-01  3.288e-02
## neighbourhood_simplifiedCrown Heights      3.635e-03  2.603e-02
## neighbourhood_simplifiedDitmars Steinway    7.489e-03  3.963e-02
## neighbourhood_simplifiedEast Elmhurst      -1.036e-01  4.869e-02
## neighbourhood_simplifiedEast Flatbush      -3.250e-02  3.413e-02
## neighbourhood_simplifiedEast Harlem         1.082e-01  2.776e-02
## neighbourhood_simplifiedEast New York      -7.710e-02  4.364e-02
## neighbourhood_simplifiedEast Village        3.957e-01  2.526e-02
## neighbourhood_simplifiedElmhurst          -1.629e-01  3.917e-02
## neighbourhood_simplifiedFinancial District  2.574e-01  3.318e-02
## neighbourhood_simplifiedFlatbush          -7.189e-02  3.348e-02
## neighbourhood_simplifiedFlushing           1.195e-02  3.451e-02
## neighbourhood_simplifiedFort Greene        2.002e-01  3.588e-02
## neighbourhood_simplifiedGowanus            2.026e-01  4.494e-02
## neighbourhood_simplifiedGramercy           3.476e-01  4.081e-02
## neighbourhood_simplifiedGreenpoint         1.402e-01  2.725e-02
## neighbourhood_simplifiedGreenwich Village  5.614e-01  3.981e-02
## neighbourhood_simplifiedHarlem             7.348e-02  2.364e-02
## neighbourhood_simplifiedHell's Kitchen      3.460e-01  2.493e-02
## neighbourhood_simplifiedInwood            -1.597e-01  4.393e-02
## neighbourhood_simplifiedJackson Heights   -1.595e-01  4.102e-02
## neighbourhood_simplifiedJamaica            6.734e-02  4.644e-02
## neighbourhood_simplifiedKips Bay           2.871e-01  3.486e-02
## neighbourhood_simplifiedLong Island City    1.413e-01  3.308e-02
## neighbourhood_simplifiedLower East Side     2.912e-01  2.870e-02
## neighbourhood_simplifiedMidtown            4.464e-01  2.598e-02
## neighbourhood_simplifiedMorningside Heights 1.009e-01  3.972e-02
## neighbourhood_simplifiedMurray Hill        2.977e-01  3.655e-02
## neighbourhood_simplifiedNolita             4.625e-01  4.253e-02
## neighbourhood_simplifiedOther              6.086e-02  2.174e-02
## neighbourhood_simplifiedPark Slope         2.452e-01  3.514e-02
## neighbourhood_simplifiedProspect Heights    1.471e-01  4.049e-02

```

## neighbourhood_simplifiedProspect-Lefferts Gardens	-5.057e-02	3.461e-02
## neighbourhood_simplifiedRidgewood	-1.459e-01	3.662e-02
## neighbourhood_simplifiedSoHo	5.918e-01	3.872e-02
## neighbourhood_simplifiedSouth Slope	1.551e-01	4.089e-02
## neighbourhood_simplifiedSunnyside	-2.840e-02	3.682e-02
## neighbourhood_simplifiedSunset Park	-9.784e-02	3.720e-02
## neighbourhood_simplifiedTheater District	4.645e-01	3.841e-02
## neighbourhood_simplifiedUpper East Side	3.145e-01	2.539e-02
## neighbourhood_simplifiedUpper West Side	3.545e-01	2.497e-02
## neighbourhood_simplifiedWashington Heights	-6.080e-02	2.947e-02
## neighbourhood_simplifiedWest Village	4.857e-01	3.119e-02
## neighbourhood_simplifiedWilliamsburg	2.103e-01	2.268e-02
## neighbourhood_simplifiedWoodside	-3.275e-01	3.488e-02
## property_type_simplifiedEntire condominium	1.941e-01	1.787e-02
## property_type_simplifiedEntire guest suite	-5.978e-02	3.662e-02
## property_type_simplifiedEntire house	7.627e-02	1.847e-02
## property_type_simplifiedEntire loft	2.866e-01	2.234e-02
## property_type_simplifiedEntire serviced apartment	1.217e-01	3.548e-02
## property_type_simplifiedEntire townhouse	1.346e-01	2.268e-02
## property_type_simplifiedOther	-3.970e-02	4.071e-02
## property_type_simplifiedPrivate room in apartment	-2.298e-01	4.952e-02
## property_type_simplifiedPrivate room in condominium	-1.807e-01	5.455e-02
## property_type_simplifiedPrivate room in house	-2.033e-01	5.094e-02
## property_type_simplifiedPrivate room in loft	-1.182e-01	5.609e-02
## property_type_simplifiedPrivate room in townhouse	-4.115e-01	5.214e-02
## property_type_simplifiedRoom in boutique hotel	1.332e-01	5.801e-02
## property_type_simplifiedRoom in hotel	1.226e-01	5.912e-02
## property_type_simplifiedShared room in apartment	9.319e-02	6.003e-02
## room_typeHotel room	-1.392e-02	6.043e-02
## room_typePrivate room	-2.747e-01	4.916e-02
## room_typeShared room	-7.783e-01	5.555e-02
## accommodates	9.163e-02	2.418e-03
## bedrooms	1.113e-01	5.766e-03
## minimum_nights	-2.576e-04	1.035e-04
## maximum_nights	3.439e-08	1.574e-08
## host_total_listings_count	2.939e-04	2.041e-05
## host_is_superhostt	9.843e-03	7.134e-03
## host_identity_verifiedt	-3.566e-02	6.900e-03
## instant_bookablet	-3.446e-02	6.240e-03
## latitude	2.205e-01	8.933e-02
## longitude	-8.685e-01	8.406e-02
##	t value	Pr(> t)
## (Intercept)	-8.463	< 2e-16 ***
## neighbourhood_simplifiedBedford-Stuyvesant	-0.838	0.401829
## neighbourhood_simplifiedBushwick	-2.870	0.004104 **
## neighbourhood_simplifiedCarroll Gardens	6.456	1.09e-10 ***
## neighbourhood_simplifiedChelsea	16.129	< 2e-16 ***
## neighbourhood_simplifiedChinatown	6.868	6.67e-12 ***
## neighbourhood_simplifiedClinton Hill	7.166	7.93e-13 ***
## neighbourhood_simplifiedCrown Heights	0.140	0.888946
## neighbourhood_simplifiedDitmars Steinway	0.189	0.850133
## neighbourhood_simplifiedEast Elmhurst	-2.127	0.033410 *
## neighbourhood_simplifiedEast Flatbush	-0.952	0.340911
## neighbourhood_simplifiedEast Harlem	3.897	9.76e-05 ***

## neighbourhood_simplifiedEast New York	-1.767	0.077311	.
## neighbourhood_simplifiedEast Village	15.666	< 2e-16	***
## neighbourhood_simplifiedElmhurst	-4.159	3.20e-05	***
## neighbourhood_simplifiedFinancial District	7.759	8.90e-15	***
## neighbourhood_simplifiedFlatbush	-2.147	0.031769	*
## neighbourhood_simplifiedFlushing	0.346	0.729110	
## neighbourhood_simplifiedFort Greene	5.582	2.40e-08	***
## neighbourhood_simplifiedGowanus	4.507	6.59e-06	***
## neighbourhood_simplifiedGramercy	8.518	< 2e-16	***
## neighbourhood_simplifiedGreenpoint	5.145	2.70e-07	***
## neighbourhood_simplifiedGreenwich Village	14.100	< 2e-16	***
## neighbourhood_simplifiedHarlem	3.108	0.001886	**
## neighbourhood_simplifiedHell's Kitchen	13.878	< 2e-16	***
## neighbourhood_simplifiedInwood	-3.636	0.000278	***
## neighbourhood_simplifiedJackson Heights	-3.888	0.000101	***
## neighbourhood_simplifiedJamaica	1.450	0.147046	
## neighbourhood_simplifiedKips Bay	8.237	< 2e-16	***
## neighbourhood_simplifiedLong Island City	4.270	1.96e-05	***
## neighbourhood_simplifiedLower East Side	10.146	< 2e-16	***
## neighbourhood_simplifiedMidtown	17.181	< 2e-16	***
## neighbourhood_simplifiedMorningside Heights	2.540	0.011081	*
## neighbourhood_simplifiedMurray Hill	8.146	3.94e-16	***
## neighbourhood_simplifiedNolita	10.875	< 2e-16	***
## neighbourhood_simplifiedOther	2.800	0.005119	**
## neighbourhood_simplifiedPark Slope	6.979	3.05e-12	***
## neighbourhood_simplifiedProspect Heights	3.633	0.000281	***
## neighbourhood_simplifiedProspect-Lefferts Gardens	-1.461	0.143920	
## neighbourhood_simplifiedRidgewood	-3.984	6.80e-05	***
## neighbourhood_simplifiedSoHo	15.286	< 2e-16	***
## neighbourhood_simplifiedSouth Slope	3.793	0.000149	***
## neighbourhood_simplifiedSunnyside	-0.771	0.440524	
## neighbourhood_simplifiedSunset Park	-2.630	0.008540	**
## neighbourhood_simplifiedTheater District	12.095	< 2e-16	***
## neighbourhood_simplifiedUpper East Side	12.384	< 2e-16	***
## neighbourhood_simplifiedUpper West Side	14.194	< 2e-16	***
## neighbourhood_simplifiedWashington Heights	-2.063	0.039107	*
## neighbourhood_simplifiedWest Village	15.571	< 2e-16	***
## neighbourhood_simplifiedWilliamsburg	9.275	< 2e-16	***
## neighbourhood_simplifiedWoodside	-9.389	< 2e-16	***
## property_type_simplifiedEntire condominium	10.864	< 2e-16	***
## property_type_simplifiedEntire guest suite	-1.632	0.102663	
## property_type_simplifiedEntire house	4.131	3.63e-05	***
## property_type_simplifiedEntire loft	12.826	< 2e-16	***
## property_type_simplifiedEntire serviced apartment	3.430	0.000605	***
## property_type_simplifiedEntire townhouse	5.936	2.96e-09	***
## property_type_simplifiedOther	-0.975	0.329483	
## property_type_simplifiedPrivate room in apartment	-4.641	3.48e-06	***
## property_type_simplifiedPrivate room in condominium	-3.312	0.000928	***
## property_type_simplifiedPrivate room in house	-3.991	6.60e-05	***
## property_type_simplifiedPrivate room in loft	-2.108	0.035060	*
## property_type_simplifiedPrivate room in townhouse	-7.892	3.09e-15	***
## property_type_simplifiedRoom in boutique hotel	2.297	0.021636	*
## property_type_simplifiedRoom in hotel	2.074	0.038049	*
## property_type_simplifiedShared room in apartment	1.552	0.120556	

```

## room_typeHotel room                -0.230 0.817849
## room_typePrivate room              -5.588 2.33e-08 ***
## room_typeShared room               -14.011 < 2e-16 ***
## accommodates                       37.901 < 2e-16 ***
## bedrooms                           19.310 < 2e-16 ***
## minimum_nights                     -2.488 0.012845 *
## maximum_nights                     2.185 0.028880 *
## host_total_listings_count           14.404 < 2e-16 ***
## host_is_superhostt                 1.380 0.167671
## host_identity_verifiedt            -5.168 2.39e-07 ***
## instant_bookablet                 -5.523 3.36e-08 ***
## latitude                           2.469 0.013566 *
## longitude                          -10.332 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4446 on 26356 degrees of freedom
## Multiple R-squared:  0.5861, Adjusted R-squared:  0.5849
## F-statistic: 478.5 on 78 and 26356 DF,  p-value: < 2.2e-16

# predict on test data
lm_pred_log <- predict(lm_model, newdata = test_data)

# convert log price back to price
lm_pred_price <- exp(lm_pred_log)
actual_price <- test_data$price
actual_log_price <- test_data$log_price

# evaluation metrics for linear regression model
lm_rmse_log <- sqrt(mean((actual_log_price - lm_pred_log)^2))
lm_mae_log <- mean(abs(actual_log_price - lm_pred_log))

lm_rmse_price <- sqrt(mean((actual_price - lm_pred_price)^2))
lm_mae_price <- mean(abs(actual_price - lm_pred_price))

lm_r2 <- 1 - sum((actual_log_price - lm_pred_log)^2) /
  sum((actual_log_price - mean(actual_log_price))^2)

lm_results <- data.frame(
  Model = "Linear Regression",
  RMSE_Log = lm_rmse_log,
  MAE_Log = lm_mae_log,
  RMSE_Price = lm_rmse_price,
  MAE_Price = lm_mae_price,
  R_Squared = lm_r2
)
lm_results

##           Model  RMSE_Log  MAE_Log RMSE_Price MAE_Price R_Squared
## 1 Linear Regression 0.4430137 0.3399476   77.81849  43.71618 0.5687459

```

Regression Tree

```

library(rpart)
library(rpart.plot)

```

```

tree_model <- rpart(
  log_price ~ neighbourhood_simplified + property_type_simplified +
    room_type + accommodates + bedrooms + minimum_nights +
    maximum_nights + host_total_listings_count +
    host_is_superhost + host_identity_verified +
    instant_bookable + latitude + longitude,
  data = train_data,
  method = "anova"
)
tree_model

## n= 26435
##
## node), split, n, deviance, yval
##      * denotes terminal node
##
## 1) root 26435 12587.62000 4.579768
##    2) property_type_simplified=Other,Private room in apartment,Private room in condominium,Private r
##      4) neighbourhood_simplified=Astoria,Bedford-Stuyvesant,Bushwick,Crown Heights,Ditmars Steinway,
##        8) host_total_listings_count>=13.5 660    90.10253 3.488718 *
##        9) host_total_listings_count< 13.5 9363   2054.45000 4.084064
##          18) accommodates< 3.5 8869   1702.07800 4.055442 *
##          19) accommodates>=3.5 494    214.65610 4.597941 *
##      5) neighbourhood_simplified=Carroll Gardens,Chelsea,Chinatown,Clinton Hill,East Harlem,East Vil
##    3) property_type_simplified=Entire apartment,Entire condominium,Entire guest suite,Entire house,E
##      6) bedrooms< 1.5 7977   1762.40500 4.888752
##        12) neighbourhood_simplified=Astoria,Bedford-Stuyvesant,Bushwick,Carroll Gardens,Crown Heights
##        13) neighbourhood_simplified=Chelsea,Chinatown,Clinton Hill,East Village,Financial District,Fo
##      7) bedrooms>=1.5 5184   1639.92000 5.228690
##        14) longitude>=-73.96264 2816    753.88730 5.074592 *
##        15) longitude< -73.96264 2368    739.64270 5.411942 *

# predict
tree_pred_log <- predict(tree_model, newdata = test_data)
tree_pred_price <- exp(tree_pred_log)

# evaluate
tree_rmse_log <- sqrt(mean((actual_log_price - tree_pred_log)^2))
tree_mae_log <- mean(abs(actual_log_price - tree_pred_log))

tree_rmse_price <- sqrt(mean((actual_price - tree_pred_price)^2))
tree_mae_price <- mean(abs(actual_price - tree_pred_price))

tree_r2 <- 1 - sum((actual_log_price - tree_pred_log)^2) /
  sum((actual_log_price - mean(actual_log_price))^2)

tree_results <- data.frame(
  Model = "Regression Tree",
  RMSE_Log = tree_rmse_log,
  MAE_Log = tree_mae_log,
  RMSE_Price = tree_rmse_price,
  MAE_Price = tree_mae_price,
  R_Squared = tree_r2
)

```

```
tree_results
```

```
##           Model  RMSE_Log  MAE_Log RMSE_Price MAE_Price R_Squared
## 1 Regression Tree 0.4747412 0.3659188   84.02092  47.30104 0.5047634
```

Random Forest

```
# random forest
library(randomForest)
```

```
set.seed(123)
```

```
# fit
rf_model <- randomForest(
  log_price ~ neighbourhood_simplified + property_type_simplified +
    room_type + accommodates + bedrooms + minimum_nights +
    maximum_nights + host_total_listings_count +
    host_is_superhost + host_identity_verified +
    instant_bookable + latitude + longitude,
  data = train_data,
  ntree = 100,
  mtry = 4,
  importance = TRUE
)
rf_model
```

```
##
## Call:
## randomForest(formula = log_price ~ neighbourhood_simplified +      property_type_simplified + room_
##           Type of random forest: regression
##           Number of trees: 100
## No. of variables tried at each split: 4
##
##           Mean of squared residuals: 0.1533054
##           % Var explained: 67.8
```

```
# predict
rf_pred_log <- predict(rf_model, newdata = test_data)
rf_pred_price <- exp(rf_pred_log)

# evaluate
rf_rmse_log <- sqrt(mean((actual_log_price - rf_pred_log)^2))
rf_mae_log <- mean(abs(actual_log_price - rf_pred_log))

rf_rmse_price <- sqrt(mean((actual_price - rf_pred_price)^2))
rf_mae_price <- mean(abs(actual_price - rf_pred_price))

rf_r2 <- 1 - sum((actual_log_price - rf_pred_log)^2) /
  sum((actual_log_price - mean(actual_log_price))^2)

rf_results <- data.frame(
  Model = "Random Forest",
  RMSE_Log = rf_rmse_log,
  MAE_Log = rf_mae_log,
```

```

    RMSE_Price = rf_rmse_price,
    MAE_Price = rf_mae_price,
    R_Squared = rf_r2
  )
rf_results

##           Model  RMSE_Log  MAE_Log RMSE_Price MAE_Price R_Squared
## 1 Random Forest 0.3891996 0.2902902   69.88159   38.21714 0.6671537

# variable importance in random forest model
rf_model$importance

##                                     %IncMSE  IncNodePurity
## neighbourhood_simplified 0.137085132   1286.42035
## property_type_simplified 0.149945503   2817.96602
## room_type                0.087971707   1669.02216
## accommodates             0.108205068   1316.29565
## bedrooms                 0.049314380    775.48662
## minimum_nights           0.016446355    289.64710
## maximum_nights           0.015585567    372.89957
## host_total_listings_count 0.058902805    697.36990
## host_is_superhost         0.004570313     85.51203
## host_identity_verified    0.003668624    108.24301
## instant_bookable          0.008525575    118.47258
## latitude                  0.072293297   1061.49162
## longitude                 0.111256005   1260.85375

```

Actual vs. Predicted Plots

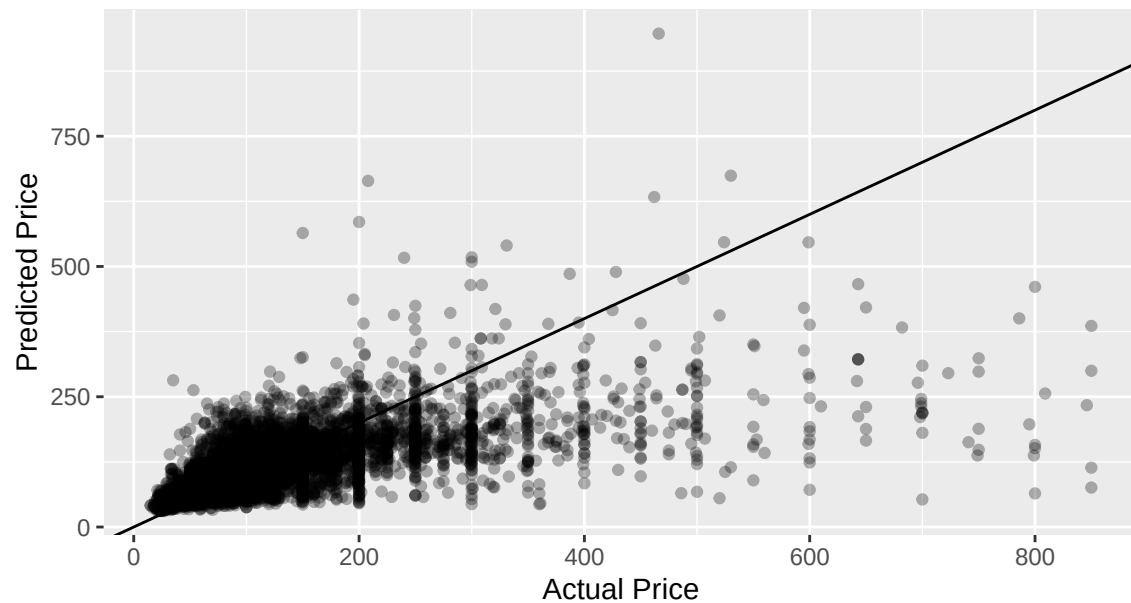
```

# linear regression
lm_pred_df <- data.frame(
  actual_price = actual_price,
  predicted_price = lm_pred_price
)

ggplot(lm_pred_df, aes(x = actual_price, y = predicted_price)) +
  geom_point(alpha = 0.3) +
  geom_abline(slope = 1, intercept = 0) +
  labs(
    title = "Linear Regression: Actual vs Predicted Airbnb Prices",
    x = "Actual Price",
    y = "Predicted Price"
  )

```

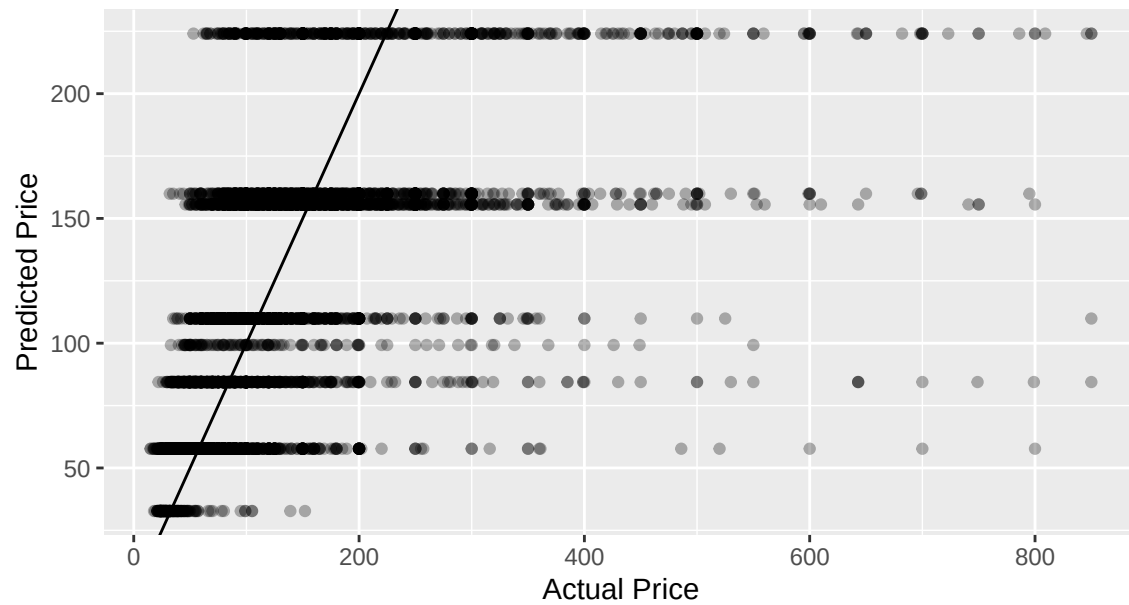
Linear Regression: Actual vs Predicted Airbnb Prices



```
# regression tree
tree_pred_df <- data.frame(
  actual_price = actual_price,
  predicted_price = tree_pred_price
)

ggplot(tree_pred_df, aes(x = actual_price, y = predicted_price)) +
  geom_point(alpha = 0.3) +
  geom_abline(slope = 1, intercept = 0) +
  labs(
    title = "Regression Tree: Actual vs Predicted Airbnb Prices",
    x = "Actual Price",
    y = "Predicted Price"
  )
```

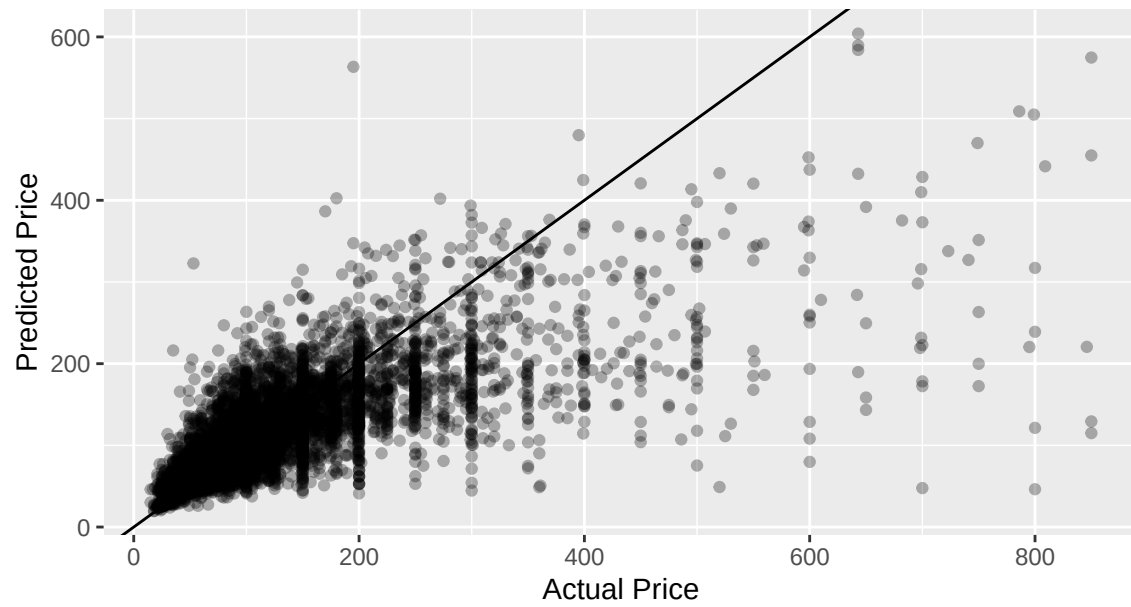
Regression Tree: Actual vs Predicted Airbnb Prices



```
# random forest
rf_pred_df <- data.frame(
  actual_price = actual_price,
  predicted_price = rf_pred_price
)

ggplot(rf_pred_df, aes(x = actual_price, y = predicted_price)) +
  geom_point(alpha = 0.3) +
  geom_abline(slope = 1, intercept = 0) +
  labs(
    title = "Random Forest: Actual vs Predicted Airbnb Prices",
    x = "Actual Price",
    y = "Predicted Price"
  )
```

Random Forest: Actual vs Predicted Airbnb Prices



Model Comparison

##	Model	RMSE_Log	MAE_Log	RMSE_Price	MAE_Price	R_Squared
## 1	Linear Regression	0.4430137	0.3399476	77.81849	43.71618	0.5687459
## 2	Regression Tree	0.4747412	0.3659188	84.02092	47.30104	0.5047634
## 3	Random Forest	0.3891996	0.2902902	69.88159	38.21714	0.6671537